

Revision 02 • The Inference Stack

Interactive revision of [lesson 02](#): each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 • Autoscaling on the wrong signal PARTIAL

Covers: 02, 18

QUESTION

Four replicas of Llama 3 8B sit behind an autoscaler that adds a replica when CPU utilization passes 70%. During a spike, users wait 5+ seconds for the first token, but CPU shows 12% on every server, so nothing scales. What's wrong, and what should the autoscaler watch instead?

Answer

- **CPU is the wrong signal.** The GPU does the work, so the CPU sits mostly idle even when a replica is overloaded.
- **Watch two engine signals instead:** 1. **Queue length** (requests waiting, or time spent waiting): the 5 seconds is mostly time in the engine's queue, because prefill itself takes milliseconds. 2. **KV cache usage:** each user holds its own cache, so a full cache means no room for more users. Use the engine's metric, not raw GPU memory: engines reserve ~90% of GPU memory at startup, so `nvidia-smi` always looks "full".
- **Scale early, on the trend:** a new replica must load 16 GB of weights before it serves anyone.

RE-DRILL

When a latency number looks huge, ask where the time goes: compute (prefill/decode) or waiting (queue).

Q2 • How many users fit on one GPU? SOLID

Covers: 02, 18, 19

QUESTION

One H100 (80 GB) serves Llama 3 8B in BF16 (16 GB of weights). Ignoring runtime overhead, how many tokens of KV cache fit (128 KB per token), and how many 4,096-token conversations can run at once?

How I solved it

1. Memory left for the cache: $80 - 16 = 64 \text{ GB}$ (the weights take 16).

2. Tokens: $64 \times 10^9 \text{ bytes} \div 128 \times 10^3 \text{ bytes} = 0.5 \times 10^6 \approx \text{500k tokens}$ (488k with 128 KB = 131,072 bytes).

3. Users: $500 \times 10^3 \div 4,096 \approx \text{122 users}$ (~119 with the exact numbers).

Caveat: this is the naive upper bound. Runtime, activations and headroom take ~11 GB more, so a real engine gets ~400k tokens (lesson 18).

Q3 · Kernel launch overhead PARTIAL

Covers: 02, 17, 18

QUESTION

One decode step launches ~300 kernels at $\sim 4 \mu\text{s}$ of CPU time each; the ideal step (bytes $\div$ bandwidth) is 4.8 ms. How much time goes to launches, as a share of the step, and how do you cut it?

Answer

- Launch time: $300 \times 4 \mu\text{s} = 1,200 \mu\text{s} = \text{1.2 ms}$, which is $1.2 \div 4.8 = \text{25\%}$ of the ideal step.
- **Fix 1, kernel fusion:** merge several kernels into one, so there are fewer launches and fewer round trips to memory.
- **Fix 2, CUDA graphs:** record the step's launches once and replay them with a single launch, so ~300 CPU launches become ~1. Needs fixed tensor shapes (lesson 17: preallocated KV cache) and costs 1–5 GB of memory (lesson 18).

RE-DRILL

Percentages: divide the part by the whole, then sanity-check with a simple fraction ($\frac{1}{4}$, $\frac{1}{3}$, $\frac{1}{2}$). Remember the pair "fusion + CUDA graphs".

Q4 · Cold start of a new replica SOLID

Covers: 02

QUESTION

During a spike the autoscaler launches a new replica, which must load 16 GB of weights from storage at $\sim 2 \text{ GB/s}$. How long until it serves, what does that mean for when to trigger, and how can you shorten it?

Answer

- **Load time:** $16 \text{ GB} \div 2 \text{ GB/s} = \text{8 s}$ of weight loading alone. Add container start, GPU init and CUDA graph capture, and real cold starts are often tens of seconds.
- **When to trigger:** early, on a rising trend, e.g. **KV cache usage at 70–80%** or **queue length growing**, so the new replica is ready before the current ones saturate.

- **How to shorten it:**
- Faster source: keep weights in **host RAM**, or on a **local NVMe cache** instead of network storage.
- Fewer bytes: **FP8 weights** (8 GB) halve the load to ~ 4 s.
- Don't load at spike time: keep a **warm standby pool** of replicas already loaded.

RE-DRILL

For any "time = bytes $\div$ speed" problem, the levers are always: fewer bytes, faster speed, or do it ahead of time.

Exercise · Round-robin vs prefix-aware router SOLID

Covers: 02

QUESTION

12 requests, each a shared system prompt (A, B, C cycling, 1,000 tokens) plus a fresh 50-token user message, go to 4 replicas that cache prompts they've prefilled. How many prompt tokens does the fleet prefill under round-robin ($i \% 4$) vs prefix-aware routing ($\text{prefix_id} \% 4$)?

Setup I needed clarified first: `policy` exists to compare two routers with one function; `prefix_id` is which system prompt (not which replica); under round-robin the cycles $i \% 4$ and $i \% 3$ don't line up, so every replica sees A, B and C and caches nothing useful.

The structure:

1. **State:** one set per replica, holding the prompt ids it has cached.
2. **Pick the replica:** round-robin $i \% n$; prefix-aware $\text{prefix_id} \% n$.
3. **Cost:** `user_len` always, plus `prefix_len` only if that replica hasn't cached the prompt.
4. **Update:** add the prompt to that replica's set.
5. **Output:** sum of costs.

```
def total_prefill_tokens(requests, n_replicas, policy):
    cached = [set() for _ in range(n_replicas)] # Step 1 · state per replica
    total = 0
    for i, (prefix_id, prefix_len, user_len) in enumerate(requests):
        if policy == "round_robin": # Step 2 · pick the replica
            replica = i % n_replicas
        else:
            replica = prefix_id % n_replicas
        cost = user_len + (0 if prefix_id in cached[replica] else prefix_len) # Step 3
        cached[replica].add(prefix_id) # Step 4 · update state
        total += cost
    return total # Step 5

requests = [(i % 3, 1000, 50) for i in range(12)]
total_prefill_tokens(requests, 4, "round_robin") # 12,600 (12 prompt prefills)
total_prefill_tokens(requests, 4, "prefix") # 3,600 (3 prompt prefills)
```

Result: prefix-aware routing prefills **3.5x fewer** tokens.

Caveat (interview follow-up): pure prefix routing left replica 3 idle while 0–2 took all the load. Real inference-aware routers combine both: send to the replica holding the prefix unless it's overloaded, otherwise the least-loaded one.